how to connect ONVIF camera in c++?

Thought for 12s >

The most common way to connect to an ONVIF camera in C++ is to use **gSOAP** and generate C/C++ client bindings from the ONVIF WSDL files. ONVIF devices expose SOAP web services, and the **Device service is the entry point to the other ONVIF services**. Cameras also commonly support **WS-Discovery** for discovery on the local network, and clients can use **GetCapabilities** to learn which services the camera actually supports. ONVIF +3

A practical connection flow looks like this:

1. **Discover or know the camera IP**
   - Either use WS-Discovery on the LAN, or start with a known camera IP. ONVIF discovery is part of the standard discovery model. ONVIF +1
2. **Get the device service endpoint**
   - Many cameras expose something like: `http://CAMERA_IP/onvif/device_service`
3. **Authenticate**
   - ONVIF cameras often require username/password, frequently using WS-Security UsernameToken or HTTP auth depending on the operation and device configuration. The ONVIF device-management WSDL and core spec include security/access-control support, and gSOAP's ONVIF examples are built for this style of client. ONVIF +2
4. **Call** `GetCapabilities`
   - This tells you whether the device supports media, PTZ, imaging, events, and other services. ONVIF explicitly recommends querying capabilities instead of assuming a service exists. ONVIF +1
5. **Use the returned service URLs**
   - For video streaming, you typically call the media service to get a stream URI such as RTSP. Profile T covers video streaming over IP networks. ONVIF +1

Best option in C++: gSOAP

gSOAP provides ONVIF examples and documents the workflow for generating C/C++ code from ONVIF WSDLs with `wsdl2h` and `soapcpp2`. Genivia's ONVIF example specifically describes a C++ ONVIF client flow. Genivia +2

Typical setup:

```bash
# 1) Generate header from ONVIF WSDLs
wsdl2h -o onvif.h -t typemap.dat https://www.onvif.org/ver10/device/wsdl/d

# 2) Generate C/C++ proxy/stub code
```

Bash

After generation, you create a proxy object for the Device service and call methods such as `GetCapabilities`.

Minimal example structure

This is a simplified example of what the code usually looks like with gSOAP-generated classes:

```cpp
#include "soapDeviceBindingProxy.h"
#include "wsseapi.h"   // gSOAP WS-Security plugin
#include "onvif.nsmap"

int main() {
    DeviceBindingProxy device;

    // Optional timeout settings
    device.soap->recv_timeout = 10;
    device.soap->send_timeout = 10;
    device.soap->connect_timeout = 10;

    // Add WS-Security UsernameToken
    soap_register_plugin(device.soap, soap_wsse);
    soap_wsse_add_UsernameTokenDigest(device.soap, nullptr, "admin", "pass

    _tds__GetCapabilities req;
    _tds__GetCapabilitiesResponse resp;

    // Ask for all categories, if supported by your generated bindings
    req.Category.push_back(tt__CapabilityCategory__All);
```

```
    const char* deviceXAddr = "http://192.168.1.100/onvif/device_service";

    if (device.GetCapabilities(deviceXAddr, nullptr, &req, resp) == SOAP_O
        // Inspect resp.Capabilities
        // resp.Capabilities->Media->XAddr, etc.
    } else {
        device.soap_stream_fault(std::cerr);
    }

    device.destroy();
```

‹/› C++

Then get the stream URL

After `GetCapabilities`, use the Media service endpoint from the returned capabilities and request a stream URI. In practice the flow is:

- Device service → `GetCapabilities`
- Media service → `GetProfiles`
- Media service → `GetStreamUri`

That usually gives you an RTSP URL you can open with FFmpeg, OpenCV, GStreamer, or VLC-compatible code. ONVIF profiles define the service sets required for streaming devices, and Profile T is the modern profile for video streaming features. ONVIF +1

Common problems

- **Wrong endpoint**: use the camera's actual ONVIF device-service URL, not just the web UI URL.
- **Auth mismatch**: some cameras need WS-Security UsernameToken; others may also rely on HTTP auth/TLS depending on config. ONVIF +1
- **Assuming services exist**: always call `GetCapabilities` first. Optional services are not guaranteed. ONVIF +1
- **Clock skew**: digest-based auth can fail if camera time is badly off.
- **Local discovery blocked**: WS-Discovery uses multicast and may not cross VLANs/subnets cleanly. ONVIF discovery is standardized, but network topology can still interfere. ONVIF +1

Easiest architecture

A very practical stack is:

- **gSOAP** for ONVIF SOAP calls

- **FFmpeg / OpenCV / GStreamer** for the RTSP video after you get the stream URI

That keeps ONVIF for control and RTSP for media.

If you want, I can give you a **full working C++ example** for:

1. `GetCapabilities` ,
2. `GetProfiles` ,
3. `GetStreamUri` ,
4. then opening the RTSP stream with OpenCV.

Sources

give me the logical flow for cursor

Here is a clean **logical flow** you can give to Cursor for building **ONVIF camera connection in C++**.

Logical flow for Cursor

Build a C++ ONVIF client that connects to an ONVIF camera, authenticates, queries capabilities, gets media profiles, retrieves the RTSP stream URI, and optionally opens the stream with OpenCV.

1. Project goal

Implement a C++ module that:

- connects to an ONVIF camera
- authenticates with username/password
- calls ONVIF Device Service
- fetches service capabilities
- fetches media profiles
- gets RTSP stream URI
- exposes clean reusable functions/classes
- optionally opens the RTSP stream using OpenCV

---

2. Main architecture

Create the following modules:

`OnvifClient`

Responsible for:

- SOAP client initialization
- WS-Security username token setup
- calling ONVIF services
- handling endpoint URLs

`OnvifDeviceService`

Responsible for:

- `GetCapabilities`
- retrieving media service URL
- retrieving PTZ/imaging/event service URLs if needed

`OnvifMediaService`

Responsible for:

- `GetProfiles`
- `GetStreamUri`

`CameraConnectionInfo`

Store:

- camera IP / host
- device service URL
- username
- password
- media service URL
- selected profile token
- stream URI

`RtspStreamPlayer` optional

Responsible for:

- opening RTSP URI using OpenCV
- reading frames
- exposing reconnect/error handling

3. Functional flow

Step 1: Input camera info

Accept:

- camera IP address
- ONVIF device service URL
  example: `http://192.168.1.100/onvif/device_service`
- username
- password

Step 2: Initialize SOAP client

Set up gSOAP proxy/client:

- register WS-Security plugin
- configure connect/send/recv timeout
- configure namespaces
- attach username token digest

Step 3: Connect to Device Service

Call:

- `GetCapabilities`

Expected result:

- camera confirms ONVIF communication
- response includes URLs for supported services

Extract:

- media service XAddr
- PTZ service XAddr if available
- imaging service XAddr if available

If this fails:

- report authentication failure
- report endpoint failure
- report SOAP/network failure

Step 4: Connect to Media Service

Using the `Media->XAddr` returned from `GetCapabilities`:

Call:

- `GetProfiles`

Expected result:

- list of media profiles
- each profile has a token

Choose:

- first valid profile
- or allow selection if multiple profiles exist

Store chosen profile token

Step 5: Request stream URI

Call Media Service:

- `GetStreamUri`

Use:

- selected profile token
- stream setup = RTP-Unicast
- transport protocol = RTSP

Expected result:

- RTSP URI string

Store:

- `rtsp://...`

Step 6: Return connection result

Return a structured response object containing:

- connection success/failure
- capabilities

- media service URL
- selected profile token
- stream URI
- error message if failed

Step 7: Optional video open

If OpenCV integration is enabled:

- create `cv::VideoCapture`
- open the RTSP URI
- read frames in loop
- handle reconnect if stream drops

---

4. Error handling flow

Implement clear failure stages:

Network errors

- camera unreachable
- timeout
- wrong IP / wrong URL

Authentication errors

- invalid username/password
- WS-Security failure
- unsupported auth mode

Protocol errors

- invalid SOAP response
- ONVIF service not supported
- missing media service capability

Media errors

- no profiles returned
- invalid profile token
- stream URI not returned

RTSP playback errors

- RTSP URI valid but stream cannot open
- codec/network issue

Each function should return:

- `bool success`
- error code enum
- human-readable error message

---

5. Recommended class flow

High-level call chain

```
main()
 └── OnvifClient::connect()
      ├── initializeSoap()
      ├── authenticate()
      ├── getCapabilities()
      │    └── extractMediaXAddr()
      ├── getProfiles()
      │    └── selectProfileToken()
      ├── getStreamUri()
      └── return CameraSessionInfo
```

Optional:

```
main()
 └── RtspStreamPlayer::open(streamUri)
      ├── create VideoCapture
      ├── read frames
      └── display/process frames
```

---

6. Suggested folder structure

```
onvif-camera-cpp/
├── CMakeLists.txt
├── third_party/
│   └── gsoap/
├── generated/
```

```
│   ├── soapC.cpp
│   ├── soapClient.cpp
│   ├── soapDeviceBindingProxy.cpp
│   ├── soapMediaBindingProxy.cpp
│   ├── soapStub.h
│   └── ...
├── include/
│   ├── onvif_client.hpp
│   ├── onvif_device_service.hpp
│   ├── onvif_media_service.hpp
│   ├── camera_connection_info.hpp
│   ├── result.hpp
│   └── rtsp_stream_player.hpp
├── src/
│   ├── onvif_client.cpp
│   ├── onvif_device_service.cpp
│   ├── onvif_media_service.cpp
│   ├── rtsp_stream_player.cpp
│   └── main.cpp
└── README.md
```

---

7. Implementation rules for Cursor

Tell Cursor to follow these rules:

- use **gSOAP-generated ONVIF bindings**
- keep ONVIF logic separate from RTSP/OpenCV logic
- use RAII for SOAP object lifecycle
- avoid putting all logic in `main.cpp`
- expose clean reusable methods
- use enums/structs for error handling
- log every major request/response step
- support future extension for PTZ commands

---

8. Minimum functions to implement

In `OnvifClient`

- `bool initialize()`
- `bool setCredentials(const std::string&, const std::string&)`
- `bool connectDevice(const std::string& deviceServiceUrl)`

- `std::string getMediaServiceUrl() const`
- `std::string getSelectedProfileToken() const`
- `std::string getStreamUri() const`

In `OnvifDeviceService`

- `Result getCapabilities(const std::string& deviceServiceUrl)`

In `OnvifMediaService`

- `Result getProfiles(const std::string& mediaServiceUrl)`
- `Result getStreamUri(const std::string& mediaServiceUrl, const std::string& profileToken)`

In `RtspStreamPlayer`

- `bool open(const std::string& rtspUri)`
- `bool read(cv::Mat& frame)`
- `void close()`

---

9. End-to-end runtime flow

```
User provides:
camera IP + username + password

↓
Build device service URL

↓
Initialize SOAP + WS-Security

↓
Call GetCapabilities

↓
Extract Media XAddr

↓
Call GetProfiles

↓
Select a profile token

↓
```

```
Call GetStreamUri

↓
Return RTSP URI

↓
Optional: open RTSP stream with OpenCV
```

---

10. Prompt you can paste directly into Cursor

```
Build a production-style C++ ONVIF client using gSOAP.

Requirements:
- Connect to ONVIF camera using device service URL
- Authenticate with username/password using WS-Security UsernameToken
- Call GetCapabilities on Device Service
- Extract Media service XAddr
- Call GetProfiles on Media Service
- Select the first valid profile token
- Call GetStreamUri and return RTSP URI
- Organize code into reusable classes:
  - OnvifClient
  - OnvifDeviceService
  - OnvifMediaService
  - CameraConnectionInfo
  - optional RtspStreamPlayer using OpenCV
- Add clear error handling with error enum + message
- Use RAII and avoid putting everything in main.cpp
- Produce:
  1. folder structure
  2. header files
  3. cpp files
  4. CMakeLists.txt
  5. sample main.cpp
- Keep ONVIF control logic separate from RTSP playback logic
- Make the code easy to extend later for PTZ support
```

If you want, I can turn this into a **much more detailed Cursor task spec** with:

- exact file-by-file responsibilities
- class definitions
- method signatures

- implementation order.

give me full working C++ example

Thought for 20s ›

Below is a **practical full example** for a C++ ONVIF client using **gSOAP** that:

- connects to an ONVIF camera
- authenticates with **WS-Security UsernameToken**
- calls **GetCapabilities**
- reads the **Media service URL**
- calls **GetProfiles**
- selects the first profile
- calls **GetStreamUri**
- prints the RTSP URL

This follows the standard ONVIF flow: use the **Device service** to get service endpoints, then use the **Media service** to list profiles and request a streaming URI. ONVIF's official media WSDL includes `GetProfiles` and `GetStreamUri`, and ONVIF's application guide describes the same service pattern. gSOAP also provides ONVIF examples and generated bindings workflow. Genivia +3

1) Project layout

```
onvif_cpp_example/
├── CMakeLists.txt
├── generated/
│   ├── ... gSOAP-generated files ...
├── include/
│   └── onvif_camera_client.hpp
└── src/
    ├── main.cpp
    └── onvif_camera_client.cpp
```

2) Generate the gSOAP ONVIF bindings

You need the ONVIF/gSOAP generated code first. gSOAP's ONVIF example documents generating code from ONVIF WSDLs with `wsdl2h` and `soapcpp2`. ONVIF publishes the official device and media WSDLs. Genivia +1

Example generation flow:

```bash
mkdir -p generated
cd generated

# Example: create a combined ONVIF header.
# Adjust paths for your gSOAP installation.
wsdl2h \
  -o onvif.h \
  -t /path/to/gsoap/typemap.dat \
  https://www.onvif.org/ver10/device/wsdl/devicemgmt.wsdl \
  https://www.onvif.org/ver10/media/wsdl/media.wsdl

soapcpp2 \
  -j -CL -x \
```

Bash

You will typically get files such as:

- `soapStub.h`
- `soapH.h`
- `soapC.cpp`
- `soapClient.cpp`
- `DeviceBindingProxy.h/.cpp`
- `MediaBindingProxy.h/.cpp`
- `wsdd.nsmap` or `onvif.nsmap`-style namespace map depending on generation output

Important: the exact generated filenames can vary slightly depending on your gSOAP version/options. The code below assumes common proxy names like `DeviceBindingProxy` and `MediaBindingProxy`. If your generated class names differ, replace them accordingly. That variation comes from gSOAP codegen details, not ONVIF itself. Genivia

3) Header: `include/onvif_camera_client.hpp`

```
#pragma once

#include <string>
#include <vector>
#include <optional>

struct OnvifResult {
    bool ok = false;
    std::string message;
};

struct OnvifProfileInfo {
    std::string token;
    std::string name;
};

struct OnvifConnectionInfo {
    std::string device_service_url;
    std::string media_service_url;
    std::vector<OnvifProfileInfo> profiles;
    std::string selected_profile_token;
    std::string stream_uri;
};

class OnvifCameraClient {
public:
    OnvifCameraClient(
        std::string deviceServiceUrl,
        std::string username,
        std::string password);

    OnvifResult connect();
    const OnvifConnectionInfo& info() const;

private:
    OnvifResult getCapabilities();
    OnvifResult getProfiles();
    OnvifResult getStreamUri();

    void configureSoap(struct soap* soapCtx) const;

private:
    std::string deviceServiceUrl_;
    std::string username_;
```

```
    std::string password_;
```

‹/› C++

---

4) Source: `src/onvif_camera_client.cpp`

Update the include names if your generated filenames differ.

```cpp
#include "onvif_camera_client.hpp"

#include <iostream>
#include <sstream>

// gSOAP generated headers
#include "soapH.h"
#include "DeviceBindingProxy.h"
#include "MediaBindingProxy.h"
#include "wsseapi.h"
#include "onvif.nsmap"

OnvifCameraClient::OnvifCameraClient(
    std::string deviceServiceUrl,
    std::string username,
    std::string password)
    : deviceServiceUrl_(std::move(deviceServiceUrl)),
      username_(std::move(username)),
      password_(std::move(password)) {
    info_.device_service_url = deviceServiceUrl_;
}

const OnvifConnectionInfo& OnvifCameraClient::info() const {
    return info_;
}

void OnvifCameraClient::configureSoap(struct soap* soapCtx) const {
    soap_register_plugin(soapCtx, soap_wsse);

    // Basic timeouts
    soapCtx->connect_timeout = 10;
    soapCtx->send_timeout = 10;
    soapCtx->recv_timeout = 10;

    // Optional socket flags can be configured here if needed.
    // soapCtx->socket_flags = MSG_NOSIGNAL;
```

```
}

OnvifResult OnvifCameraClient::connect() {
    auto r1 = getCapabilities();
    if (!r1.ok) {
        return r1;
    }

    auto r2 = getProfiles();
    if (!r2.ok) {
        return r2;
    }

    auto r3 = getStreamUri();
    if (!r3.ok) {
        return r3;
    }

    return {true, "Connected successfully"};
}

OnvifResult OnvifCameraClient::getCapabilities() {
    DeviceBindingProxy device;
    configureSoap(device.soap);

    if (soap_wsse_add_UsernameTokenDigest(
            device.soap, nullptr, username_.c_str(), password_.c_str()) !=
        return {false, "Failed to add WS-Security UsernameTokenDigest"};
    }

    _tds__GetCapabilities req;
    _tds__GetCapabilitiesResponse resp;

    req.Category.push_back(tt__CapabilityCategory__All);

    int rc = device.GetCapabilities(
        deviceServiceUrl_.c_str(),
        nullptr,
        &req,
        resp);

    if (rc != SOAP_OK) {
        std::ostringstream oss;
        oss << "GetCapabilities failed. SOAP fault:\n";
        device.soap_stream_fault(oss);
        return {false, oss.str()};
    }

    if (!resp.Capabilities) {
```

```
        return {false, "GetCapabilities returned no Capabilities object"};
    }

    if (!resp.Capabilities->Media) {
        return {false, "Camera does not report Media capability"};
    }

    if (resp.Capabilities->Media->XAddr.empty()) {
        return {false, "Media XAddr is empty"};
    }

    info_.media_service_url = resp.Capabilities->Media->XAddr;
    return {true, "GetCapabilities succeeded"};
}

OnvifResult OnvifCameraClient::getProfiles() {
    if (info_.media_service_url.empty()) {
        return {false, "Media service URL is empty"};
    }

    MediaBindingProxy media;
    configureSoap(media.soap);

    if (soap_wsse_add_UsernameTokenDigest(
            media.soap, nullptr, username_.c_str(), password_.c_str()) !=
        return {false, "Failed to add WS-Security UsernameTokenDigest"};
    }

    _trt__GetProfiles req;
    _trt__GetProfilesResponse resp;

    int rc = media.GetProfiles(
        info_.media_service_url.c_str(),
        nullptr,
        &req,
        resp);

    if (rc != SOAP_OK) {
        std::ostringstream oss;
        oss << "GetProfiles failed. SOAP fault:\n";
        media.soap_stream_fault(oss);
        return {false, oss.str()};
    }

    if (resp.Profiles.empty()) {
        return {false, "No media profiles returned by camera"};
    }

    info_.profiles.clear();
```

```
    for (const auto* profile : resp.Profiles) {
        if (!profile) {
            continue;
        }

        OnvifProfileInfo p;
        p.token = profile->token;
        p.name = profile->Name;
        info_.profiles.push_back(std::move(p));
    }

    if (info_.profiles.empty()) {
        return {false, "Profiles response contained no usable profiles"};
    }

    info_.selected_profile_token = info_.profiles.front().token;
    return {true, "GetProfiles succeeded"};
}

OnvifResult OnvifCameraClient::getStreamUri() {
    if (info_.media_service_url.empty()) {
        return {false, "Media service URL is empty"};
    }

    if (info_.selected_profile_token.empty()) {
        return {false, "Selected profile token is empty"};
    }

    MediaBindingProxy media;
    configureSoap(media.soap);

    if (soap_wsse_add_UsernameTokenDigest(
            media.soap, nullptr, username_.c_str(), password_.c_str()) !=
        return {false, "Failed to add WS-Security UsernameTokenDigest"};
    }

    _trt__GetStreamUri req;
    _trt__GetStreamUriResponse resp;

    req.ProfileToken = info_.selected_profile_token;

    // Build StreamSetup
    req.StreamSetup = soap_new_tt__StreamSetup(media.soap);
    req.StreamSetup->Stream = tt__StreamType__RTP_Unicast;
    req.StreamSetup->Transport = soap_new_tt__Transport(media.soap);
    req.StreamSetup->Transport->Protocol = tt__TransportProtocol__RTSP;

    int rc = media.GetStreamUri(
        info_.media_service_url.c_str(),
```

```
        nullptr,
        &req,
        resp);

    if (rc != SOAP_OK) {
        std::ostringstream oss;
        oss << "GetStreamUri failed. SOAP fault:\n";
        media.soap_stream_fault(oss);
        return {false, oss.str()};
    }

    if (!resp.MediaUri) {
        return {false, "GetStreamUri returned null MediaUri"};
    }

    info_.stream_uri = resp.MediaUri->Uri;
    if (info_.stream_uri.empty()) {
        return {false, "GetStreamUri returned empty URI"};
    }
```

‹/› C++

---

5) Source: `src/main.cpp`

```
#include "onvif_camera_client.hpp"

#include <iostream>

int main(int argc, char** argv) {
    if (argc < 4) {
        std::cerr << "Usage:\n"
                  << "  " << argv[0]
                  << " <device_service_url> <username> <password>\n\n"
                  << "Example:\n"
                  << "  " << argv[0]
                  << " http://192.168.1.100/onvif/device_service admin 123
        return 1;
    }

    const std::string deviceServiceUrl = argv[1];
    const std::string username = argv[2];
    const std::string password = argv[3];
```

```cpp
    OnvifCameraClient client(deviceServiceUrl, username, password);

    const auto result = client.connect();
    if (!result.ok) {
        std::cerr << "ERROR: " << result.message << "\n";
        return 2;
    }

    const auto& info = client.info();

    std::cout << "Device Service URL : " << info.device_service_url << "\n
    std::cout << "Media Service URL  : " << info.media_service_url << "\n"
    std::cout << "Profiles:\n";

    for (const auto& p : info.profiles) {
        std::cout << "  - token=" << p.token << ", name=" << p.name << "\n
    }

    std::cout << "Selected Profile   : " << info.selected_profile_token <<
    std::cout << "RTSP Stream URI    : " << info.stream_uri << "\n";
```

‹/› C++

---

6) `CMakeLists.txt`

You need to point this at your local gSOAP installation and generated files.

```
cmake_minimum_required(VERSION 3.16)
project(onvif_cpp_example LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Adjust these paths
set(GSOAP_INCLUDE_DIR "/path/to/gsoap")
set(GSOAP_IMPORT_DIR "/path/to/gsoap/import")
set(GSOAP_PLUGIN_DIR "/path/to/gsoap/plugin")

include_directories(
    include
    generated
```

```
    ${GSOAP_INCLUDE_DIR}
    ${GSOAP_IMPORT_DIR}
    ${GSOAP_PLUGIN_DIR}
)

add_executable(onvif_example
    src/main.cpp
    src/onvif_camera_client.cpp

    generated/soapC.cpp
    generated/soapClient.cpp
    generated/stdsoap2.cpp
    generated/DeviceBindingProxy.cpp
    generated/MediaBindingProxy.cpp

    # wsse plugin source from gSOAP
    ${GSOAP_PLUGIN_DIR}/wsseapi.c
    ${GSOAP_PLUGIN_DIR}/smdevp.c
    ${GSOAP_PLUGIN_DIR}/mecevp.c
    ${GSOAP_PLUGIN_DIR}/threads.c
)

# OpenSSL is commonly required by wsse plugin
find_package(OpenSSL REQUIRED)

target_link_libraries(onvif_example
    PRIVATE
    OpenSSL::SSL
    OpenSSL::Crypto
```

‹/› cmake

---

7) Build

```bash
mkdir build
cd build
```

‹/› Bash

Run:

‹/› Bash

Bash

---

8) Optional: open the RTSP stream with OpenCV

ONVIF `GetStreamUri` gives you the stream URI; the media service spec explicitly defines `GetStreamUri` as the operation used to request a valid RTSP or HTTP stream URI for a media profile. After that, opening the URI is normally done by a media stack such as OpenCV/FFmpeg/GStreamer, not by ONVIF itself. ONVIF +1

Example:

```cpp
#include <opencv2/opencv.hpp>
#include <iostream>

int main() {
    std::string rtsp = "rtsp://user:pass@192.168.1.100:554/Streaming/Chann
    cv::VideoCapture cap(rtsp, cv::CAP_FFMPEG);

    if (!cap.isOpened()) {
        std::cerr << "Failed to open RTSP stream\n";
        return 1;
    }

    cv::Mat frame;
    while (true) {
        if (!cap.read(frame) || frame.empty()) {
            std::cerr << "Failed to read frame\n";
            break;
        }

        cv::imshow("ONVIF RTSP", frame);
        if (cv::waitKey(1) == 27) { // ESC
            break;
        }
    }
```

C++

---

9) Important notes

Some cameras expose **Media ver10**, some also expose **Media2 ver20**. The example above uses the classic **ver10 media service** because `GetProfiles` and `GetStreamUri` are defined there and remain widely supported. ONVIF also publishes Media2 specifications and WSDLs, but many real-world cameras still expose the ver10 media endpoint, so `GetCapabilities -> Media XAddr -> GetProfiles -> GetStreamUri` is still a very practical baseline. ONVIF +3

Also, some cameras are picky about authentication and time sync. If UsernameToken digest auth fails even with correct credentials, check camera time/NTP and try plain UsernameToken if your device expects that mode instead. ONVIF's core ecosystem supports secure authentication patterns around these services, but vendor behavior can still differ. ONVIF +1

10) Honest caveat

This is a **working pattern** and the code structure is correct for a standard gSOAP + ONVIF setup, but I cannot verify the exact generated class/type names in your local gSOAP output from here because those filenames can vary slightly by generation options and environment. The most likely edits you may need are:

- `DeviceBindingProxy.h/.cpp` vs generated proxy name on your machine
- `MediaBindingProxy.h/.cpp` vs generated proxy name on your machine
- namespace map filename
- plugin source list in CMake
- the exact generated response member naming if your gSOAP version differs slightly

If you want, I can give you the **next version adapted to your exact generated files** if you paste:

- your `generated/` file list
- your `onvif.h`
- or the first 100 lines of `soapStub.h` and the generated proxy filenames.

Sources

what is gSOAP?

gSOAP is a **C/C++ toolkit for building and consuming web services**, especially those based on **SOAP (XML-based protocols)**.

In simple terms:

> 👉 **gSOAP = a code generator + runtime library that lets your C/C++ program talk to SOAP-based services (like ONVIF cameras) without manually writing XML.**

🧠 Why gSOAP exists

Protocols like **SOAP / ONVIF** are:

- XML-heavy
- verbose
- strict (schemas, namespaces, WSDL, etc.)

Writing this manually in C++ would be painful.

👉 gSOAP solves this by:

- generating C/C++ classes from WSDL
- handling XML serialization/deserialization
- managing HTTP/SOAP communication
- supporting WS-Security (used by ONVIF cameras)

⚙️ What gSOAP actually does

1. Code generation (the magic part)

You give it a **WSDL file** (service definition), and it generates:

```
WSDL → C/C++ classes + client stubs
```

Tools:

- `wsdl2h` → converts WSDL → C/C++ header
- `soapcpp2` → generates implementation code

---

2. Runtime engine

gSOAP provides:

- HTTP client/server
- XML parser
- SOAP envelope builder
- WS-Security (username/password, digest)

---

3. What you write in C++

Instead of writing XML like this:

```xml
<soap:Envelope>
  <soap:Body>
    <GetCapabilities>...</GetCapabilities>
```

XML

You write:

C++

👉 gSOAP handles everything behind the scenes.

📦 What files gSOAP generates

After running gSOAP, you get:

```
soapStub.h          → data structures (types)
soapH.h             → service definitions
soapC.cpp           → serialization logic
soapClient.cpp      → client implementation
DeviceBindingProxy  → ONVIF Device client
MediaBindingProxy   → ONVIF Media client
```

These act like a **typed API client**.

🔥 Why gSOAP is used for ONVIF

ONVIF cameras:

- use **SOAP**
- use **WSDL-defined APIs**
- require **WS-Security (UsernameToken)**

👉 gSOAP is one of the **best tools in C++** for this because:

- native C/C++
- mature + widely used in embedded / CCTV systems
- supports ONVIF specs well
- has built-in security support

🧩 How gSOAP fits in ONVIF flow

```
[Your C++ code]
      ↓
[gSOAP client classes]
      ↓
[SOAP XML over HTTP]
      ↓
[ONVIF Camera]
```

🆚 Without gSOAP vs With gSOAP

❌ Without gSOAP

You must:

- build XML manually
- handle namespaces
- parse XML responses
- implement HTTP
- implement WS-Security

👉 extremely painful

✅ With gSOAP

```
device.GetCapabilities(...)
```

‹/› C++

👉 clean, typed, maintainable

🧠 Mental model

Think of gSOAP as:

> **"OpenAPI/Swagger client generator — but for SOAP — in C++"**

🧪 When to use gSOAP

Use it when:

- dealing with **ONVIF cameras**
- consuming **SOAP APIs**
- building **embedded systems (C/C++)**
- needing **WS-Security**

⚠️ Downsides

Be aware:

- generated code is large
- compile time can be heavy
- debugging SOAP issues can be tricky
- WSDL → code generation step adds complexity

🏁 Summary

- gSOAP = **SOAP toolkit for C/C++**
- turns WSDL → usable C++ API
- handles XML + HTTP + security
- essential for ONVIF camera integration in C++

🚀 If you want next step

I can show you:

1. **Visual architecture of ONVIF + gSOAP (clean diagram)**
2. **How WS-Security actually works under the hood**
3. **Alternative approach (no gSOAP, using pure HTTP + XML)**
4. **How to build ONVIF support into your trading system's microservices (Rust + Python + C++)**

Just tell me 👍

yes please

Thought for 10s ›

Great. Here are the **3 next-step views** in a practical way.

1. Visual architecture: ONVIF + gSOAP

```
[C++ App]
   |
   |  typed C++ calls
   v
[gSOAP-generated client proxies]
   |
   |  SOAP/XML over HTTP or HTTPS
   v
[ONVIF Camera Device Service]
   |
   |-- GetCapabilities --> returns service endpoints
   |
   +--> [Media Service]
   |       |-- GetProfiles
   |       |-- GetStreamUri --> RTSP URL
   |
   +--> [PTZ Service]
```

```
    |       |-- ContinuousMove
    |       |-- Stop
    |
    +--> [Imaging Service]
    |       |-- GetImagingSettings
    |
    +--> [Events Service]
            |-- PullMessages / Subscribe
```

What gSOAP does in this picture:

- turns WSDL files into **C/C++ types and proxy classes**
- serializes your request structs into **SOAP XML**
- sends them over HTTP/HTTPS
- parses the SOAP XML response back into **C/C++ objects**
- supports plugins for **WS-Security, WS-Addressing, and WS-Discovery** Genivia +1

For ONVIF specifically, the normal application flow is to first communicate with the **Device service**, then call **GetCapabilities** to learn which services the camera supports and which service URLs to use next. The ONVIF application guide also describes using **GetProfiles** in the media service when working with media profiles. ONVIF +1

2. How WS-Security works under the hood

When you connect to many ONVIF cameras, you do not just send username/password as plain app parameters. Instead, the client often sends a **WS-Security UsernameToken** inside the SOAP header. gSOAP supports this with its WS-Security plugin. Genivia +1

Conceptually:

```
SOAP Envelope
 ├── Header
 │    └── WS-Security
 │         └── UsernameToken
 │              ├── Username
 │              ├── PasswordDigest or PasswordText
 │              ├── Nonce
 │              └── Created timestamp
 └── Body
      └── ONVIF operation
           └── GetCapabilities / GetProfiles / GetStreamUri
```

Typical digest mode idea:

```
PasswordDigest = Base64( SHA1( Nonce + Created + Password ) )
```

Why it exists:

- avoids sending the raw password in the simplest possible form
- helps prevent replay when nonce + timestamp are validated
- fits the ONVIF SOAP security model commonly used by cameras

Why cameras sometimes fail auth even with the correct password:

- camera clock is wrong
- nonce/timestamp validation is strict
- camera expects PasswordText instead of digest
- camera vendor implemented ONVIF imperfectly

In gSOAP, this usually becomes a plugin registration plus one call that adds the UsernameToken before the SOAP method call. gSOAP's plugin system is documented in its user guide, and ONVIF examples are provided by Genivia. Genivia +1

3. Alternative approach without gSOAP

You can do ONVIF **without** gSOAP, but it is much more manual.

Option A: pure HTTP + XML in C++

Stack:

- `libcurl` for HTTP/HTTPS
- tinyxml2 / pugixml for XML building and parsing

Flow:

1. manually build SOAP XML string
2. manually add namespaces
3. manually build WS-Security header
4. send POST request to `/onvif/device_service`
5. parse XML response
6. extract media XAddr
7. repeat for media operations

Example shape:

```
libcurl POST
  Content-Type: application/soap+xml
  Body:
    <soap:Envelope>
```

```
    <soap:Header>...</soap:Header>
    <soap:Body>
      <tds:GetCapabilities>...</tds:GetCapabilities>
    </soap:Body>
  </soap:Envelope>
```

Pros:

- fewer generated files
- full control
- can be lightweight for very small use cases

Cons:

- much more error-prone
- namespaces are annoying
- WS-Security is tedious
- harder to maintain

Option B: skip ONVIF and use RTSP directly

If all you need is video:

- use the camera's RTSP URL directly
- open it via OpenCV / FFmpeg / GStreamer

Pros:

- simplest for streaming only
- no SOAP at all

Cons:

- no standard way to query profiles/capabilities
- no PTZ/imaging/events through ONVIF
- vendor-specific RTSP paths are often inconsistent

4. Recommended choice

For your use case in C++:

- use **gSOAP** if you need real ONVIF support:
  - capabilities
  - profiles
  - stream URI
  - PTZ
  - events
  - imaging

- use **OpenCV or FFmpeg** only after ONVIF gives you the RTSP URI

That split is the cleanest architecture:

- **ONVIF/gSOAP for control**
- **RTSP/OpenCV for media**

5. Practical end-to-end flow

```
User config
```



```
Initialize gSOAP
  └── register WS-Security plugin

Call Device.GetCapabilities
  └── extract Media XAddr

Call Media.GetProfiles
  └── choose profile token

Call Media.GetStreamUri
  └── receive RTSP URL

Open RTSP URL
  └── OpenCV / FFmpeg / GStreamer
```

6. What `wsdl2h` and `soapcpp2` really do

Genivia's ONVIF example and gSOAP docs describe this generation flow:

- `wsdl2h` consumes WSDL/XSD and generates a C/C++ interface header
- `soapcpp2` consumes that header and generates the client/server source code
- `typemap.dat` is used to control XML schema type mappings and namespace bindings
- gSOAP provides plugins for WS-* protocols, including security-related ones Genivia +2

So the mental model is:

```
WSDL/XSD
   ↓
wsdl2h
   ↓
C/C++ interface header
   ↓
soapcpp2
```

```
    ↓
proxy classes + serializers + SOAP runtime glue
```

7. Best way to tell Cursor what to build

Paste this:

```
Build an ONVIF camera client in C++ using gSOAP.

Architecture:
- OnvifClient: orchestration
- DeviceServiceClient: GetCapabilities
- MediaServiceClient: GetProfiles, GetStreamUri
- optional PtzServiceClient
- RtspPlayer: OpenCV-based stream reader

Rules:
- use gSOAP generated proxies from ONVIF WSDLs
- use WS-Security UsernameToken
- first call Device.GetCapabilities
- extract Media XAddr from capabilities
- then call Media.GetProfiles
- choose the first valid profile token
- then call Media.GetStreamUri
- return the RTSP URI
- keep ONVIF control logic separate from RTSP playback
- use clean error enum and error messages
- structure code into headers and cpp files
- provide CMakeLists.txt
- make the design extensible for PTZ and events
```

8. My recommendation for you

If your goal is to get a camera stream and later support PTZ, snapshots, and events, start with this milestone order:

1. `GetCapabilities`
2. `GetProfiles`
3. `GetStreamUri`
4. open RTSP with OpenCV
5. add PTZ
6. add event subscription

That keeps the project small and debuggable.

I can next give you a **file-by-file production-ready C++ project skeleton** for this flow.

Sources

give me the download link for gSOAP

Here are the **official download links for gSOAP** (safe and commonly used):

🔽 Official gSOAP download (recommended)

👉 Download gSOAP Toolkit (official site) ↗

- Maintained by Genivia (original developers)
- Contains latest versions, docs, and install instructions
- Recommended starting point

🔽 Direct download (SourceForge)

👉 Download gSOAP Toolkit ZIP (latest release) ↗

- Direct `.zip` download
- Open-source (GPLv2 license)
- Cross-platform (Linux, Windows, macOS, embedded systems) SourceForge

---

🔽 Project page (browse versions)

👉 gSOAP project page (all versions) ↗

- View all releases
- Check latest updates
- Download specific versions if needed SourceForge

🔽 GitHub resources (examples & docs)

👉 gSOAP GitHub repository (examples & tutorials) ↗

- Contains:

  - ONVIF examples
  - tutorials
  - documentation
- Not the primary binary download, but very useful for development GitHub

---

⚠️ Which one should YOU use?

For your ONVIF C++ project:

👉 **Use this combo:**

1. Download toolkit from **SourceForge (ZIP)**
2. Read docs from **Genivia site**
3. Use examples from **GitHub**

⚡ Quick install (Linux)

After downloading:

```bash
tar -xvf gsoap_2.x.x.tar.gz
cd gsoap-2.x.x

./configure
make -j
sudo make install
```

Or just use source files directly (recommended for CMake projects):

- `gsoap/stdsoap2.cpp`
- `gsoap/plugin/wsseapi.c`

🚀 Pro tip (important)

For ONVIF:

- You **don't need to install system-wide**
- Just:
  - copy `gsoap/` folder into your project

- include headers
- compile with your project

👉 This is how most production systems do it (especially embedded + microservices)

If you want, I can next give you:

✅ exact **minimal gSOAP setup for ONVIF (copy-paste ready)**
✅ or a **Docker environment for building your ONVIF C++ client**

Sources